

BÁO CÁO DỰ ÁN: HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN

Tài liệu Thiết kế Hệ thống (Enterprise-Grade Online Auction System)

TÓM TẮT DỰ ÁN (EXECUTIVE SUMMARY)

Dự án phát triển một nền tảng đấu giá thời gian thực (Real-time Auction Platform) theo mô hình phân tán Client-Server. Vượt ra khỏi các yêu cầu CRUD cơ bản, hệ thống được thiết kế để giải quyết triệt để các bài toán hóc búa trong kỹ thuật phần mềm: **Đảm bảo tính ACID trong giao dịch tài chính, tối ưu hóa độ trễ (Sub-millisecond Latency) bằng cấu trúc dữ liệu In-memory, và khả năng tự phục hồi (Fault Tolerance).**

1. Giới thiệu mục tiêu và phạm vi thực hiện

1.1. Mục tiêu cốt lõi:

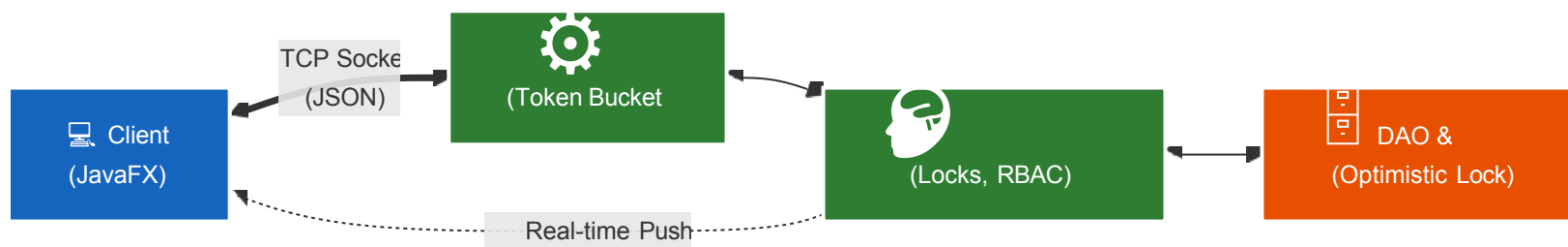
Xây dựng một hệ thống giao dịch tài chính thời gian thực minh bạch, chống gian lận và có khả năng mở rộng (Scalability). Hệ thống phải chịu tải được các kịch bản "Sniping" (hàng trăm người dùng cùng đặt giá trong những giây cuối cùng) mà không xảy ra hiện tượng thắt cổ chai (Bottleneck) hay sai lệch dòng tiền.

1.2. Phạm vi công nghệ & Nghiệp vụ:

- Client (Presentation Layer):** Ứng dụng Desktop JavaFX, kiến trúc Non-blocking UI, giao diện Glassmorphism, tích hợp Real-time Chart và OS-level Notification.
- Server (Application Layer):** Java Socket Server đa luồng, kiến trúc Event-Driven, quản lý Connection Pool (HikariCP).
- Nghiệp vụ nâng cao (Advanced Features):** Proxy Bidding (Auto-bid), Dutch Auction (Instant Resolution), Thuật toán Trending (Exponential Decay), Ban Cascade (Thu hồi giao dịch tự động), và Cơ chế Heartbeat/Reconnect.

2. Kiến trúc tổng thể hệ thống

Hệ thống được thiết kế theo kiến trúc **Multi-tier Client-Server** kết hợp **Command Dispatcher Pattern** và **Event-Driven Architecture**.



Sự tinh tế trong Thiết kế Kiến trúc (Architectural Elegance):

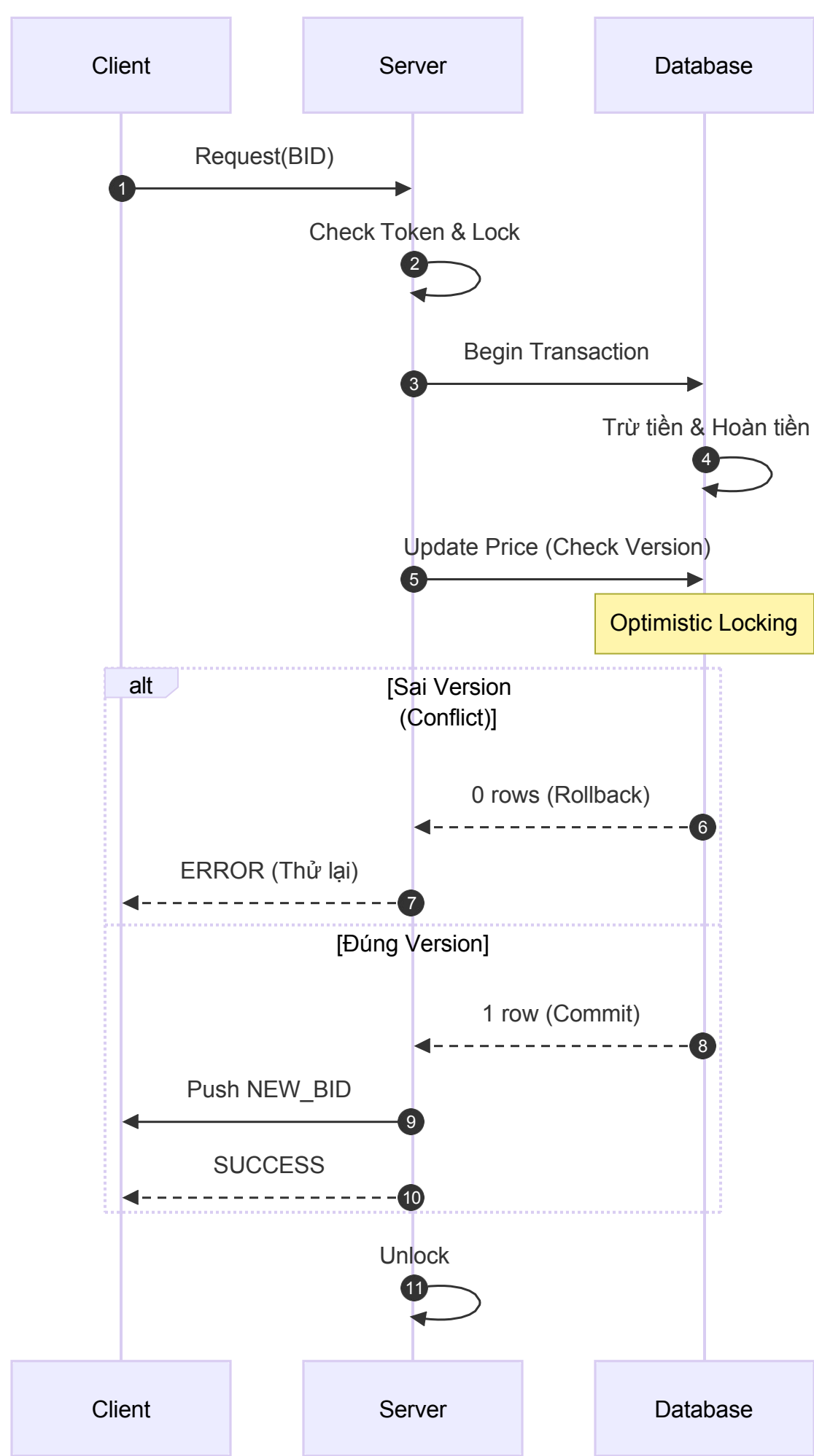
- Polymorphic Network Protocol:** Giao tiếp qua TCP Socket sử dụng Jackson JSON Serialization. Tích hợp **BasicPolymorphicTypeValidator** để ngăn chặn triệt để các cuộc tấn công Deserialization (RCE), đảm bảo an toàn khi truyền tải các Object đa hình.
- Truy vết phân tán (Distributed Tracing):** Tích hợp **MDC (Mapped Diagnostic Context)** trong Logback. Mỗi Request được gán một duy nhất reqId giúp truy vết chính xác luồng chạy của từng giao dịch xuyên suốt các Thread trong môi trường đa luồng.
- Thread-safe Broadcasting:** Sử dụng **CopyOnWriteArrayList** tại **ClientConnectionHub** để quản lý danh sách Client, đảm bảo việc Push sự kiện Real-time không bao giờ bị lỗi **ConcurrentModificationException** khi có Client ngắt kết nối đột ngột.

3. Các tính năng Advanced & Kỹ thuật cốt lõi

3.1. Xử lý đồng thời & Giao dịch nguyên tử (Concurrency & ACID)

Để giải quyết bài toán hàng trăm người dùng cùng đặt giá tại một thời điểm (Race Condition), hệ thống áp dụng cơ chế bảo vệ 3 lớp:

- Fine-grained Locking (Khóa cục bộ trên RAM):** Sử dụng **ConcurrentHashMap** tại **AuctionManager**. Chỉ khóa (Lock) cục bộ theo ID sản phẩm, giúp các phiên đấu giá khác nhau chạy song song hoàn toàn (Maximized Throughput). Tích hợp tryLock(500ms) để chống Deadlock.
- Optimistic Concurrency Control (Khóa lạc quan tại DB):** Bảng items có trường version. Lệnh cập nhật giá luôn kèm **WHERE id = ? AND version = ?**. Loại bỏ hoàn toàn hiện tượng Lost Update.
- Transactional DAO:** Các thao tác tài chính (Trừ tiền -> Hoàn tiền người cũ -> Ghi Log -> Cập nhật giá) được gom vào một **Connection** duy nhất với **setAutoCommit(false)**. Đảm bảo tính nguyên tử (Atomicity) - Rollback toàn bộ nếu có bất kỳ bước nào thất bại.



3.2. Thuật toán & Cấu trúc dữ liệu tối ưu (Algorithms & Data Structures)

- **Thuật toán Trending (Exponential Decay):** Thay vì đếm số lượng Bid thô sơ, hệ thống đánh giá độ "Hot" của sản phẩm bằng công thức suy giảm hàm mũ ($e^{-h/\tau}$). Kết hợp khối lượng Bid dài hạn (W_{long}), khối lượng đột biến (W_{short}) và hệ số đa dạng người chơi (U_{cnt}). Ngăn chặn tuyệt đối việc một User tự Spam Bid để thao túng bảng xếp hạng.
- **Cấu trúc dữ liệu Trie (Prefix Tree):** Tính năng Autocomplete không dùng được nạp lên RAM dưới dạng cây Trie (**SELECT LIKE**) gây nghẽn Disk I/O. Toàn bộ tên sản phẩm (**TrieManager**), cho phép tìm kiếm với độ phức tạp $O(L)$ (L là độ dài từ khóa), phản hồi tức thời.
- **Zero-Polling với DelayQueue:** Hệ thống chốt phiên (**SettlementService**) không dùng Timer quét Database. Thời gian kết thúc được đưa vào `java.util.concurrent.DelayQueue` . Worker Thread có độ phức tạp chờ là $O(1)$, chỉ thức dậy chính xác vào mili-giây sản phẩm hết hạn, tiết kiệm 99% tài nguyên CPU

3.3. Sự tinh tế trong xử lý Nghiệp vụ & UI (Business & UI Elegance)

- **Debouncing & Throttling:** Tại thanh tìm kiếm, hệ thống áp dụng kỹ thuật Debounce (độ trễ 300ms bằng **TimerTask**). Chỉ gửi
- Request xuống Server khi người dùng đã ngừng gõ phím, giảm thiểu tối đa lượng Request rác gây quá tải Database.

- **Tính lũy đẳng (Idempotency) trong DAO:** Các thao tác như thêm vào Watchlist hay gửi lời mời kết bạn sử dụng câu lệnh **INSERT IGNORE**. Đảm bảo dù Client có gửi trùng lặp Request do lỗi mạng, Database vẫn không sinh ra dữ liệu rác.
- **Smart Viewport Cropping:** Hình ảnh sản phẩm và Avatar tải về từ Cloudinary được xử lý cắt cúp trung tâm (Center-crop) bằng toán học (**Rectangle 2D**) ngay trên RAM trước khi render, đảm bảo UI luôn đồng nhất tỷ lệ mà không làm méo ảnh.

3.4. Các chức năng đạt được theo barem điểm

Barem đánh giá	Điểm	Chức năng đạt được	Hướng giải quyết	Lý do lựa chọn
Xác định và triển khai các lớp chính	0.5	Có các lớp User, Bidder, Seller, Admin, Item, Electronics, Art, Vehicle, Auction, BidTransaction	Tách lớp dùng chung trong module auction-shared, server/client cùng sử dụng model thống nhất	Đúng yêu cầu đề bài, tránh trùng định nghĩa dữ liệu giữa client và server
Áp dụng nguyên tắc OOP	1.0	Encapsulation, Inheritance, Polymorphism, Abstraction	Dùng abstract class Entity, User, Item; các lớp con kế thừa theo vai trò và loại sản phẩm; dữ liệu truy cập qua getter/setter	Cấu trúc rõ ràng, dễ mở rộng thêm vai trò hoặc loại sản phẩm
Áp dụng design pattern phù hợp	1.0	Factory, Strategy/Command, Observer/Event-based, Singleton-style manager	ItemFactory tạo sản phẩm theo category; AuctionBidPipeline chọn chiến lược English/Dutch; ActionRegistry điều phối request; realtime notifier push sự kiện cho client	Giảm phụ thuộc giữa các phần, dễ thêm loại đấu giá hoặc request mới
Quản lý người dùng, sản phẩm	1.0	Đăng ký, đăng nhập, OTP, phân quyền user/admin; thêm/sửa/xóa/xem/loc sản phẩm	Server xử lý qua các handler/service/DAO; MySQL lưu người dùng và sản phẩm; client JavaFX chỉ gửi request qua Socket	Đảm bảo chỉ server truy cập database, dữ liệu tập trung và dễ kiểm soát quyền
Chức năng đấu giá	1.0	Đặt giá, kiểm tra giá hợp lệ, cập nhật người dẫn đầu, quản lý ví/giao dịch	AuctionManager xử lý bid; validator kiểm tra phiên, giá và số dư; transaction DB trừ tiền, hoàn tiền, ghi log và cập nhật giá	Đảm bảo logic đấu giá đúng và không sai lệch dòng tiền
Xử lý lỗi và ngoại lệ	1.0	Chặn giá thấp hơn giá hiện tại, chặn bid khi phiên đóng, xử lý lỗi kết nối và dữ liệu	Dùng validator, exception riêng, response lỗi rõ ràng; xử lý EOFException; heartbeat/reconnect khi mất kết nối	Tăng độ ổn định, tránh crash server/client và giúp người dùng nhận lỗi dễ hiểu
Xử lý đấu giá đồng thời an toàn	1.0	Tránh lost update, rollback sai và hai người cùng thắng	Kết hợp ReentrantLock theo item, optimistic locking bằng cột version, transaction JDBC và rollback khi lỗi	Phù hợp tình huống nhiều bidder đặt giá cùng lúc, bảo toàn kết quả cuối cùng
Realtime update	0.5	Client đang xem phiên nhận bid mới/trạng thái mới ngay	Server push sự kiện qua Socket; ClientConnectionHub quản lý client bằng CopyOnWriteArrayList; client cập nhật UI qua router/notification	Không cần polling liên tục, giảm tải server và cập nhật nhanh
Thiết kế kiến trúc Client-Server	0.5	Client JavaFX, Server Socket, MySQL phía server	Client gửi JSON request qua TCP Socket; server xử lý nghiệp vụ và truy cập DAO/MySQL	Đúng yêu cầu đề bài, tách rõ giao diện, nghiệp vụ và dữ liệu
Áp dụng MVC	0.5	Client có FXML/controller/service; server có controller/handler/service/DAO/model	JavaFX controller xử lý UI; server controller nhận kết nối, handler xử lý action, DAO làm việc với database	Code dễ bảo trì, mỗi tầng có trách nhiệm riêng
Maven/Gradle, convention, mã nguồn sạch	0.5	Maven multi-module gồm auction-shared, auction-server, auction-client	Dùng Maven để quản lý dependency, build JAR; chia package theo controller/service/dao/model/ui	Build thống nhất, dễ chạy test và đóng gói
Unit Test bằng JUnit	0.5	Có test cho logic shared, bidding, concurrency, auto-bid, validation, trending	Viết JUnit ở auction-shared, auction-server, auction-client cho các logic quan trọng	Giảm lỗi hồi quy ở phần đấu giá, tính giá và validate dữ liệu
CI/CD cơ bản	0.5	Có GitHub Actions build/test tự động	Cấu hình .github/workflows/build.yml để chạy build/test khi thay đổi mã nguồn	Giúp phát hiện lỗi build/test sớm khi làm việc nhóm
Auto-Bidding	0.5	Người dùng đặt giá tối đa, hệ thống tự trả giá	Dùng AutoBidCoordinator, AutoBidRegistration, PriorityQueue và logic không vượt maxBid	Đáp ứng chức năng nâng cao, xử lý nhanh nhiều auto-bid cùng lúc
Anti-sniping	0.5	Gia hạn phiên khi có bid ở thời điểm cuối	Tích hợp kiểm tra thời điểm bid trong luồng đấu giá/chốt phiên và cập nhật lại thời gian kết thúc nếu cần	Tránh người dùng thắng bằng cách đặt giá sát giờ kết thúc
Bid History Visualization	0.5	Biểu đồ đường giá realtime trong màn hình chi tiết phiên	Client dùng JavaFX LineChart, cập nhật dữ liệu khi nhận bid mới từ server	Người dùng theo dõi diễn biến giá trực quan, không cần refresh
Tính năng sáng tạo bổ sung	0.5	Dutch Auction, Trending Lots, Trie Autocomplete, Watchlist, Chat, Rating, Notification	Tách thành service/handler/DAO riêng; dùng Trie, DelayQueue, công thức trending và notification center	Tăng trải nghiệm người dùng và thể hiện thêm thuật toán/cấu trúc dữ liệu nâng cao

Nhìn chung, hệ thống đã hoàn thành các chức năng chính của một sàn đấu giá trực tuyến: tài khoản, phân quyền, sản phẩm, đặt giá, ví tiền, lịch sử, thông báo realtime và quản trị. Ngoài ra, hệ thống có thêm các phần nâng cao như xử lý đồng thời, giao dịch ACID, auto-bid, Dutch Auction, Trie autocomplete, DelayQueue và chống spam để đảm bảo dữ liệu chính xác và phản hồi nhanh.

4. Phân chia công việc và Đóng góp của các thành viên

Dự án được chia thành 4 mảng chuyên môn hóa. Mỗi thành viên hoạt động như một kỹ sư Full-stack, chịu trách nhiệm từ khâu thiết kế kiến trúc, lựa chọn thuật toán đến triển khai thực tế cho các module được giao. Tỷ lệ hoàn thành của toàn bộ các thành viên đều đạt **100%**.

4.1. Dương Nguyên Khánh – System Architect & Core Infrastructure

Đảm nhiệm thiết kế hạ tầng mạng, tối ưu hóa thuật toán và xây dựng các cơ chế chống chịu lỗi cho Server.

- Tối ưu hóa & Thuật toán:** Xây dựng công cụ tìm kiếm Autocomplete với độ trễ cực thấp ($O(L)$) bằng cấu trúc dữ liệu **Trie**; Phát triển luồng Proxy Bidding (Auto-bid) sử dụng **PriorityQueue** để xử lý các vòng lặp đấu thầu trên RAM.
- Quản lý tài nguyên & ACID:** Tối ưu hóa cơ chế chốt phiên bằng Zero-Polling Settlement thông qua **DelayQueue**; Đảm bảo tính toàn vẹn dữ liệu với **Transactional DAO**.
- Chống chịu lỗi (Resilience):** Bảo vệ hệ thống khỏi DDoS/Spam bằng Rate Limiting (**Token Bucket**); Xây dựng cơ chế Fault Tolerance (Heartbeat & Auto-Reconnect) giúp Client tự phục hồi phiên làm việc khi rớt mạng; Quản lý vòng đời ứng dụng với **Graceful Shutdown**.
- Sự tinh tế:** Tích hợp **MDC Logging** để truy vết Request phân tán; Quản lý luồng sự kiện an toàn với **CopyOnWriteArrayList**.

4.2. Nguyễn Bá Nin – Concurrency & Advanced Business Logic

Đảm nhiệm xử lý các bài toán đồng thời (Concurrency), toán học hóa nghiệp vụ và các luồng logic phức hợp.

- Xử lý đồng thời (Concurrency):** Giải quyết triệt để bài toán Race Condition bằng Fine-grained Locking (**ReentrantLock**) trên RAM và
- Optimistic Locking** (cột version) dưới Database, đảm bảo không xảy ra hiện tượng Lost Update.
- Toán học hóa nghiệp vụ:** Thiết kế luồng Đấu giá ngược (Dutch Auction) với thuật toán **Instant Resolution** ($O(1)$); Phát triển thuật toán Trending Lots dựa trên hàm suy giảm mũ (**Exponential Decay**).
- Logic phức hợp:** Xây dựng cơ chế **Ban Cascade** (Hiệu ứng Domino) – tự động thu hồi giao dịch, hoàn tiền cọc và đơn hàng người chơi khi một tài khoản bị Admin khóa.
- Sự tinh tế:** Áp dụng tính lũy đẳng (**Idempotency**) trong các câu lệnh SQL; Sử dụng **AtomicLong** để đồng bộ hóa trạng thái UI khi xử lý các tác vụ mạng bất đồng bộ.

4.3. Nguyễn Trọng Nhân – Frontend Architect & Data Visualization

Đảm nhiệm kiến trúc giao diện, xử lý bất đồng bộ phía Client và trực quan hóa dữ liệu.

- Kiến trúc UI/UX:** Thiết kế giao diện Glassmorphism hiện đại; Tối ưu hóa luồng hiển thị với **Non-blocking UI** (đẩy I/O xuống
- Background Thread và cập nhật qua **Platform.runLater**), đảm bảo ứng dụng luôn mượt mà ở 60 FPS.
- Quản lý Trạng thái (State Management):** Xây dựng tính năng Watchlist với cơ chế đồng bộ UI toàn cục dựa trên RAM Cache
- (**ClientSession**), giúp các màn hình cập nhật trạng thái ngay lập tức.
- Trực quan hóa dữ liệu:** Tích hợp Real-time Price Chart (LineChart) cập nhật theo từng tick giá của thị trường; Xây dựng System Analytics với các biểu đồ thống kê doanh thu và trạng thái.
- Sự tinh tế:** Áp dụng kỹ thuật **Debouncing** (**300ms**) cho thanh tìm kiếm; Xử lý toán học **Viewport Cropping** để cắt cúp hình ảnh thông minh không làm méo tỷ lệ.

4.4. Nguyễn Anh Tuấn – Security, Auction Core Flow, Realtime Communication & Database

Đảm nhiệm bảo mật hệ thống, luồng tài chính cốt lõi, cơ sở dữ liệu và hệ sinh thái tương tác người dùng.

- Bảo mật (Security):** Triển khai bảo mật dữ liệu (Băm mật khẩu **SHA-256** tại Client, tích hợp OTP Email chạy bất đồng bộ); Thiết lập phân quyền **Role-Based Access Control (RBAC)** bằng Middleware để bảo vệ các API nhạy cảm.
- Luồng tài chính cốt lõi:** Xử lý luồng Đấu giá tiến (English Auction) và Quản lý ví tiền ảo (Escrow/Wallet) với các trạng thái dòng tiền nghiêm ngặt: Deposit, Hold, Refund, Sold.
- Hệ sinh thái tương tác:** Xây dựng hệ thống Real-time Chat (bao gồm Global Chat và Private Chat 1-1) cùng hệ thống Quản lý Bạn bè (Friendship).
- Database Setup:** Chính sửa và cài đặt cơ sở dữ liệu cho tránh hao tổn tài nguyên hệ thống và tối ưu cho việc truy vấn và chỉnh sửa dữ liệu.
- Sự tinh tế:** Tích hợp OS-level Notification (System Tray) với cơ chế **Debouncing** thông minh chống trôi thông báo; Xử lý triệt để ngoại lệ **EOFException** để dọn dẹp tài nguyên khi Client ngắt kết nối đột ngột.

